[El títol del TFG]

[El subtítol del tfg: Opcional]

TREBALL DE FI DE GRAU DE

Aniol Petit Cabarrocas

Jordi Taixés i Ventosa

Mathematical Engineering in Data Science

Curs 2025 - 2026

Universitat Pompeu Fabra Barcelona
Escola d'Enginyeria

Do or do not, there is no try - Master Yoda

Agraïments

dsds

Resum

[Resum aquí]

Abstract

[Abstract here]

Resumen

[Resumen aquí]

Preface

Contents

List of Figures

List of Tables

1 Introduction

2 Theoretical Preliminaries

2.1 Graph Theory Foundations

To computationally solve a routing problem across the European transit network, the physical infrastructure must be translated into a mathematical structure known as a graph. A graph $G = (V, E)$ is defined by a set of vertices V (or nodes) and a set of edges E that connect pairs of vertices. In the context of transportation routing, a standard graph is inherently directed and weighted.

Directed edges. Transit connections are unilateral. An edge $e = (u, v)$ indicates a valid route from node u to node v. The existence of $e = (u, v)$ does not imply the existence of a return edge $e' = (v, u)$ at the same time or cost.
Weighted edges. Each edge carries a cost, typically represented by a weight function $w(u, v)$. In our context, this weight represents temporal costs, such as the travel duration of a flight or the waiting time of a layover.

A path in this graph is a sequence of vertices where each adjacent pair is connected by a valid edge. The goal of a routing algorithm is to find a specific path that maximizes or minimizes a certain objective function (e.g., minimizing total weight or maximizing the number of unique vertices visited).

2.1.1 Time-Dependent vs. Time-Expanded Networks

A traditional static graph is insufficient for modeling public transit because connections do not exist continuously; they are strictly bound to specific departure and arrival schedules. To model this dimension of time, routing problems typically rely on one of two paradigms:

Time-Dependent Networks. The physical topology of the graph remains static (e.g., one vertex per station), but the weight of an edge $w(u, v, t)$ is a function of the departure time t. This keeps the network compact, but complicates the routing algorithms, as the cost of traversal fluctuates dynamically.
Time-Expanded Networks. Time is explicitly discretized and baked directly into the topology of the graph. A single physical station is duplicated into multiple vertices, each representing that station at a specific point in time (e.g., Station A at 10:00, Station A at 10:15). This dramatically increases the number of nodes and edges, but allows the use of standard, static routing algorithms, since every edge now possesses a fixed, immutable weight.

Because our problem operates within a strict 24-hour window using discrete, fixed transit schedules, modeling time efficiently is the central architectural challenge. To systematically evaluate the optimal data structure for this Guinness World Record routing problem, this thesis intentionally implements and

compares both paradigms: a memory-efficient time-dependent multi-graph (detailed in Chapter 4) and a structurally unrolled time-expanded network (detailed in Chapter 5).

2.1.2 Activity-on-Edge (AoE) vs. Activity-on-Node (AoN)

Within time-expanded models, the data can be structured in two distinct ways:

Activity-on-Edge (AoE). The traditional approach where vertices represent physical locations at specific times, and edges represent the actual transport activities (a train moving between locations) or waiting periods at a station.
Activity-on-Node (AoN). An inverted approach where the vertices themselves represent the specific transport activities (a flight with fixed departure and arrival times), and the edges represent the valid layovers or transfer opportunities connecting them.

Understanding the distinction between AoE and AoN is critical, as the choice of paradigm radically alters the branching factor, memory footprint, and traversability of the search space. A core objective of this research is to contrast these two paradigms. Chapter 4 evaluates an AoE time-dependent structure, while Chapter 5 introduces a structural pivot to an AoN framework, allowing for a direct comparative analysis of their computational viability on a continental scale.

2.2 Search and Optimization Algorithms

When graph networks become too large for brute-force enumeration, exact and heuristic optimization algorithms are required to traverse the search space efficiently.

2.2.1 Dynamic Programming and Label Setting

Dynamic Programming (DP) is a mathematical optimization method that solves complex problems by breaking them down into simpler, overlapping subproblems. In the context of graph routing, DP is often implemented via Label Setting algorithms. As the algorithm traverses the graph, it assigns "labels" to each node, where a label represents the accumulated cost or state of a specific path reaching that node.
In multi-objective optimization problems—where multiple conflicting criteria must be optimized simultaneously (e.g., minimizing time while maximizing profit)—a single optimal path cannot always be determined dynamically. Instead, the algorithm must maintain a Pareto Frontier at each node. The frontier is a set containing all non-dominated labels. A label L_1 mathematically "dominates" L_2 if it is equal or superior in all evaluated objectives, and strictly superior in at least one. Dominated labels are permanently discarded, while the non-dominated set is kept to spawn future branches.

2.2.2 Depth-First Search (DFS) and Branch-and-Bound

Depth-First Search (DFS) is a fundamental graph traversal algorithm that explores as far down a single branch of a tree as possible before backtracking. Because DFS only requires storing the nodes along the current active path, its memory complexity grows linearly with the depth of the search tree rather than exponentially with its breadth, making it highly memory-efficient.
To optimize DFS for finding absolute maximums or minimums, it is often paired with Branch-and-Bound (BnB), a paradigm for discrete and combinatorial optimization.

Branching. The algorithm systematically divides the search space by exploring subsequent child nodes.
Bounding. At each node, a bounding function calculates an optimistic mathematical estimate (an upper or lower bound) of the best possible solution that could be found by continuing down that branch.

If the calculated bound of a branch is worse than the best solution already found elsewhere in the search (the incumbent solution), the algorithm instantly abandons the branch. This technique, known as pruning, eliminates vast sections of the search space without requiring explicit evaluation.

2.3 Distributed Computing

When a computational problem's scale exceeds the memory or processing capacity of a single machine, distributed computing methodologies are employed to divide the task across a High-Performance Computing (HPC) cluster. An HPC cluster is a network of independent computers (nodes) that function as a single system. Access to these resources is governed by a workload manager or job scheduler, such as SLURM (Simple Linux Utility for Resource Management). The scheduler allocates specific hardware constraints, such as CPU cores and maximum RAM, and queues tasks to ensure that execution does not exceed physical memory limits, which would otherwise trigger immediate task termination.

To efficiently search massive combinatorial state spaces, algorithms frequently leverage data-level parallelism. In this paradigm, the exact same algorithm is executed concurrently across non-overlapping subsets of the initial data. In HPC environments, this is seamlessly orchestrated using job arrays. A job array takes a single execution script and duplicates it into dozens or hundreds of independent tasks, assigning each a unique environmental index. The algorithm uses this index to deterministically identify and process its specific slice of the search space, achieving massive throughput without the overhead of constant inter-node network communication.

Within those individual tasks, intra-node multiprocessing is often utilized to fully saturate the assigned CPU cores. However, launching multiple concurrent processes can lead to severe memory duplication, which is fatal for memory-intensive graph traversals. To mitigate this, concurrent architectures employ Inter-Process Communication (IPC) and shared memory. By utilizing thread-safe locks and shared variables, multiple worker processes can safely read global data structures and update shared states, such as a global incumbent bound, without triggering race conditions or redundant memory allocation.

2.4 Data Acquisition Concepts

To populate large-scale routing networks, real-world scheduling data must be extracted programmatically from remote servers. The standard mechanism for this is a REST API (Representational State Transfer Application Programming Interface). APIs provide structured endpoints where a client can send HTTP requests to retrieve specific, filtered datasets. In modern web architectures, this data is almost universally exchanged in JSON (JavaScript Object Notation), a lightweight, key-value serialization format. JSON allows nested, hierarchical data to be seamlessly transmitted over networks and deserialized directly into programmatic data structures, such as dictionaries and arrays, for immediate computational use.

However, when public APIs are unavailable or heavily restricted, data must be gathered via web scraping. Web scraping involves automating HTTP requests to target servers and parsing the resulting HTML, a markup language designed for human-readable browsers rather than machine consumption. This requires algorithmic extraction of target variables from unstructured or semi-structured Document Object Model (DOM) trees. Both API querying and web scraping are strictly governed by server-side rate limits (throttling). Consequently, data acquisition pipelines must implement pagination logic, exponential backoff strategies, and fault-tolerant parsing to ensure reliable, large-scale dataset generation without triggering server bans or connection timeouts.

3

Data Acquisition and Processing Pipeline

3.1 Sourcing Transit Data

The foundational step of this optimization problem was acquiring a comprehensive, time-accurate dataset of scheduled public transportation. The Guinness World Record guidelines mandate the use of scheduled public transport. Therefore, the data pipeline required a fusion of short-haul flight networks and high-speed rail schedules.

The geographic scope of the dataset was strictly and intentionally limited to the European continent. Europe is the only logical theater for this specific record due to its unparalleled density of sovereign states coupled with highly developed, cross-border transit infrastructure. However, extracting continental-scale transit data presents a significant financial barrier. Commercial aggregators typically charge per query, which is prohibitively expensive when building a dense, multi-country matrix. Consequently, a primary constraint of this project was to secure massive data volumes while strictly minimizing or eliminating acquisition costs. This budget constraint fundamentally dictated the technological approach for both flight and train data and imposed some challenges for acquiring high quality data.

3.1.1 Flight APIs

While the airline industry is highly standardized, access to global distribution systems (such as Amadeus or Sabre) requires expensive commercial licensing. To bypass these costs, the flight dataset was sourced using the AviationEdge API.

This platform offered a generous 1 month trial period charged at $7 that enabled access to most of the endpoints, among those the *flightsHistory* endpoint used to acquire the flight data. To efficiently target the API without exhausting query limits on irrelevant locations, an online global dataset of airports was acquired and programmatically filtered to isolate only active commercial airports within European borders. The resulting list of IATA (International Air Transport Association) codes served as the exact querying parameters.

By iterating through these European IATA codes and a specific 48-hour target date, the pipeline sent automated HTTP requests to extract all scheduled departures. The 48-hour window is required to ensure that routes starting at any point on day 1 have a full 24-hour window ahead of them to complete the record.

3.1.2 Train Data

Acquiring European rail data proved significantly more complex. Unlike aviation, the European rail network is heavily fragmented across dozens of national operators (e.g., SNCF, DB, Renfe, Trenitalia). There is no centralized, free REST API that aggregates all European train schedules; and master datasets are extremely expensive (often approaching five figures).

Because purchasing access to a unified commercial rail database was out of budget, the pipeline relied on programmatic web scraping. The target was a centralized rail search engine acting as a routing aggregator for a large portion of cross-border connections, namely the *Deutsche Bahn (DB) portal*.

Using the web scraping methodologies outlined in Chapter 2, a custom batch scraper was engineered to extract this data. Because modern transit portals heavily throttle automated requests, the script utilized *undetected-chromedriver* running in a headless Selenium environment to bypass bot-detection mechanisms. To guide the scraping process without blindly querying thousands of unviable city combinations, the pipeline leveraged an online compilation of major European rail corridors. The extraction algorithm systematically iterated through this predetermined list, simulating user queries between the station pairs in both forward and reverse directions. To ensure exhaustive daily coverage without triggering UI blocks, the scraper incrementally advanced the search time based on the last extracted departure, systematically sweeping the entire 24-hour window. Because scraping at this scale is computationally heavy and highly vulnerable to IP bans, the pipeline extracted exactly one standard weekday of train data. The algorithm operated under the safe assumption of schedule continuity, duplicating this 24-hour block to construct the necessary 48-hour window required for the record attempt.

The script then parsed the resulting unstructured HTML pages, isolated the Document Object Model (DOM) elements containing the departure, arrival, and transfer times, and serialized the extracted text into clean, incremental CSV records. This brute-force data engineering approach bypassed the lack of a unified API, successfully capturing the ground-level connectivity required for the routing algorithm.

3.2 Data Cleaning and Normalization

With the raw flight and train schedules acquired, the data existed in highly disparate, semi-structured formats across thousands of CSV and JSON files. To safely process and mutate this data at scale, a local SQLite database was instantiated as an intermediate staging environment. This approach enabled structured SQL queries to filter, group, and reconcile the massive datasets before exporting the finalized, clean records back to CSV format for the graph construction algorithm.

3.2.1 Entity Resolution and Standardization

A major challenge of aggregating multi-source European transit data is the lack of standardized naming conventions. A single physical location might appear under multiple aliases (e.g., “München Hbf”, “Munich Central”, or “München Hauptbahnhof”). If left uncorrected, the routing graph would treat these as distinct, disconnected nodes, destroying viable transfer opportunities.

To resolve this, a programmatic string normalization pipeline was implemented. This pipeline converted text to lowercase, stripped punctuation, removed generic tokens (e.g., “station”, “gare”, “stn”), and applied a hardcoded dictionary mapping to forcefully unify critical European hubs and border crossings under a single canonical name.

Additionally, although the flight API returned comprehensive historical delay metrics, these were ultimately stripped from the final dataset. Incorporating unpredictable historical delays into a time-expanded routing graph exponentially increases complexity without guaranteeing future accuracy. Consequently, the dataset was strictly normalized to rely solely on scheduled departure and arrival times to maintain a deterministic model.

3.2.2 The Midnight Rollover Problem and Absolute Minutes

To allow mathematical evaluation of layover times, all chronological data had to be converted into a continuous timeline. The target metric was *Absolute Minutes*, representing the elapsed time from $t = 0$ (midnight on the first day of the 48-hour routing window).
While flight data included explicit date strings, making absolute conversion straightforward, the scraped train data only provided localized *HH:MM* strings for intermediate stops. This introduced the *Midnight Rollover* problem: the data did not explicitly state when a train's journey crossed into the next calendar day.
To solve this, a chronological tracking algorithm was applied to the sequence of stops. By storing the absolute departure time of a route's origin, the algorithm deduced day boundaries by checking whether a subsequent stop's minute-value was numerically lower than the previous stop's value. If an arrival time was lower than its preceding departure time, 1440 minutes (24 hours) were automatically added to the timestamp, carrying the route into the next day. The algorithm also imputed missing values; if an arrival time was missing from the scraped HTML, it safely duplicated the departure time, ensuring no `NULL` values broke the final graph.

3.2.3 UTC Synchronization

Because the European continent spans multiple time zones (from UTC+0 in Portugal to UTC+2 in Finland), local transit times could not be directly compared. For example, a train departing Spain (UTC+1) at 10:00 and arriving in Portugal (UTC+0) at 09:30 would create a negative time-travel paradox if plotted in local time on a graph.
To ensure strict chronological consistency, every absolute minute timestamp was mapped to a universal UTC+0 baseline. This required building a comprehensive lookup table linking station locations to their respective timezone offsets. Since generic country-code matching is flawed for outermost regions, specific airport and station overrides were manually coded. For example, while mainland Spain is UTC+1, the Canary Islands were explicitly hardcoded to UTC+0. Similarly, vast countries like Russia required specific regional mapping. By subtracting these precise local offsets, the entire 48-hour European transit dataset was synchronized onto a single mathematically viable timeline.
Finally, a strict validation pass was executed across the normalized dataset. Any transit record where the absolute UTC arrival time was numerically lower than the absolute UTC departure time was permanently dropped, effectively filtering out upstream API scheduling errors and corrupted data points.

3.3 Spatial Mapping and Filtering

To evaluate a Guinness World Record attempt based on the number of countries visited, the routing algorithm must inherently "know" the geographical location and sovereign territory of every single node in the network. Furthermore, a raw European transit graph contains tens of thousands of minor stops (e.g., suburban commuter stations or rural villages) that drastically bloat the search space without offering viable international transfer opportunities. Consequently, a rigorous spatial mapping and filtering pipeline was engineered to reduce the graph to its most mathematically relevant hubs while ensuring 100% geographical accuracy.

3.3.1 Country Assignment and Border Resolution

While international flights explicitly land in known countries, domestic and cross-border train schedules scraped from the DB portal do not consistently label the country of intermediate stops. To dynamically resolve this, a multi-strategy geographical detection algorithm was implemented.
The algorithm evaluated each scraped station name through a descending hierarchy of spatial heuristics:

- **Regex code extraction:** Extracting explicit country codes often hidden in parentheses within the raw text (e.g., parsing (CH) as Switzerland or (NL) as the Netherlands).

- **Language pattern recognition:** Utilizing linguistic markers to confidently infer sovereign territory (e.g., the presence of "hl.n." for Czech main stations, "Glowny" for Poland, or " b.Wien" for Austrian suburbs).
- **GeoNames cross-referencing:** Querying a localized subset of the GeoNames database to map normalized station strings to the most populous European city matching that name.

Despite these programmatic layers, complex border stations frequently generated false positives (e.g., "Genève" being administratively in Switzerland despite a French linguistic profile, or border towns like Domodossola). To guarantee absolute integrity for record evaluation, an exhaustive list of manual topological corrections was hardcoded into the pipeline to forcefully overwrite erroneous geographical assignments at critical border crossings.

3.3.2 Station Filtering and Hub Identification

With countries successfully assigned, the sheer volume of total nodes remained computationally intractable. A scoring heuristic was developed to prune the network while safely preserving the core international skeleton. Every station was evaluated based on keyword presence (e.g., rewarding "Hauptbahnhof" or "Central", penalizing "Village" or "Halt") and its topological frequency across all scraped routes.

The filtering logic dictated that the first and last stops of any train were always preserved, alongside at least one high-scoring hub per traversed country. This successfully stripped away thousands of irrelevant intermediate stops without breaking the physical continuity of the routes.

To further isolate the absolute most critical transit hubs among the roughly 40 targeted countries, the pipeline leveraged Large Language Models (LLMs). By feeding national station lists into an advanced LLM, the model was instructed to intelligently identify only the most viable international transfer hubs for each specific country, keeping the final graph dense with high-value opportunities.

3.3.3 Intra-Country Transfers and Geocoding Validation

The final geographical challenge was enabling realistic human transfers between different stations within the same city (e.g., moving from a flight arriving at Paris CDG to a train departing from Paris Gare du Nord). Establishing a physical connection between two nodes requires highly precise geospatial coordinates and transit times.

Because commercial geocoding and routing APIs (such as the *Google Maps Distance Matrix API*) incur significant financial costs at scale, an innovative, multi-agent AI approach was designed. An LLM was prompted to evaluate the filtered hubs and identify which intra-city stations were actually worth connecting. A strict constraint was applied: any transfer requiring excessive transit time (e.g., 4 hours) was automatically discarded, as such a delay would mathematically destroy a 24-hour record attempt.

This process generated a highly focused list of 236 viable transfer connections, alongside AI-estimated coordinates and durations. Because LLMs are prone to geospatial hallucinations, a secondary LLM was deployed specifically to audit the first model's output. However, to eliminate any margin of error and ensure maximum academic rigor, a manual validation protocol was executed. Every single one of the 236 critical transfers was manually queried in the Google Maps UI to extract the exact walking or local-transit durations. This painstaking manual verification completely eliminated API costs while yielding a 100% accurate, human-viable transfer matrix for the final routing algorithm.

4

Algorithmic Approach I: Label Setting and Bounding Foundations

4.1 The Time-Dependent Transportation Graph

With a fully normalized, UTC-synchronized dataset of European transit schedules, the next step was constructing the mathematical environment for the routing algorithm. A standard static spatial graph $G = (V, E)$ is fundamentally inadequate for modeling scheduled public transportation, as an edge implies continuous availability.

To rigorously test different modeling paradigms, the first architectural approach evaluates the network as a time-dependent multi-graph. In this Activity-on-Edge (AoE) architecture, the vertices remain purely spatial, while the dimension of time is encoded directly into the attributes of the edges. This approach was deliberately chosen to maintain a highly compact memory footprint, avoiding the node explosion inherent to fully time-expanded models.

The graph was instantiated using the NetworkX library with the following structural rules:

- **Spatial Nodes (V):** A single vertex was created for each unique physical location. Nodes were tagged with specific attributes: their sovereign country (critical for the objective function) and their infrastructure type (airport or station).
- **Temporal Transit Edges (E):** Flights and trains were modeled as directed edges between origin and destination nodes. Because multiple trains or flights travel between the same two cities throughout the day, these edges stored dynamic temporal attributes. Instead of a single static weight, each edge contained an array of valid $(departure\_time, arrival\_time)$ pairs, formatted in absolute UTC minutes. To account for the 48-hour Guinness World Record window, train schedules (which operate on daily cycles) were programmatically duplicated, adding 1440 minutes to populate the second day of the graph.
- **Static Transfer Edges:** The manually validated intra-city connections (e.g., traversing from a city's airport to its central rail station) were integrated as standard, bidirectional static edges. Unlike transit edges, transfer edges were assigned a fixed scalar weight representing the human connection time in minutes, as they can be traversed at any point during the day.

By compressing schedules into arrays on spatial edges, this AoE architecture successfully avoided node redundancy. However, this time-dependent structure shifts the chronological burden entirely onto the search algorithm, which must dynamically iterate through edge arrays to find valid connections based

on its arrival time at any given node. This compact graph serves as the baseline environment to test the exact optimization methods detailed in the following sections.

4.2 Algorithm Evolution: From Baseline to Bounding

Because the Guinness World Record challenge is fundamentally a multi-objective optimization problem (maximizing unique countries visited while minimizing time within a strict 24-hour limit) a standard shortest-path algorithm like Dijkstra's cannot be used. Instead, the search must be modeled using Dynamic Programming (DP) via a Label-Setting algorithm (see Algorithm 1 in Appendix X).
In this paradigm, as the algorithm traverses the time-dependent multi-graph, it assigns a state "label" L to each node. A label is defined by the tuple $L = (v, t, C, start\_time)$. Here, v tracks the current node, t is the current absolute minute, C represents the mathematical set of unique countries visited so far, and $start\_time$ marks when the journey began. The cardinality of the set, denoted as $|C|$, represents the integer count of unique countries visited.
To prevent the search space from expanding infinitely, the algorithm relies on Pareto Dominance. At any given node, a new label L' is strictly dominated by an existing label L'' if the existing path arrived earlier or at the same time ($L''.t \leq L'.t$) while having visited a superset or equal set of countries ($L''.C \supseteq L'.C$), with at least one condition being strictly better. Dominated labels are instantly discarded.
However, implementing this baseline algorithm revealed a fatal flaw: the Pareto dominance conditions were too relaxed. Across a network spanning roughly 40 countries, the combinatorial permutations of visited country sets C caused an explosion of non-dominated labels, rendering the baseline approach computationally unfeasible. This initiated a rigorous, iterative evolution of the algorithm, where advanced mathematical pruning techniques were progressively introduced to control the state space.

4.2.1 Reachability and Upper Bound (UB) Pruning

The first major architectural change was introducing predictive pruning (see Algorithm 2 in Appendix X). Before expanding a label, the algorithm calculated an optimistic Upper Bound (UB) of the maximum new countries that could theoretically be reached from the current node v within the remaining time.
To make this computationally tractable without running a full sub-search at every node, time was discretized into fixed intervals, or "buckets" b. For any given node v and time bucket b, a precomputation step generated $R(v, b)$, the absolute set of all sovereign countries reachable from that location before the 24-hour clock expired. The theoretical Upper Bound was then calculated as the cardinality of the set difference between the reachable countries and the already visited countries: $UB = |R(v, b) \setminus C|$.
If the current number of countries visited plus this theoretical maximum ($|C| + UB$) was strictly less than or equal to the current global record, the path was mathematically dead and immediately pruned.

4.2.2 Time-Budgeted UB and Minimum Transit Time (MTT)

While standard UB pruning successfully removed hopeless paths, it was overly optimistic. It assumed that if 10 new countries were reachable from a node, all 10 could be visited, completely ignoring the physical time required to travel between them.
To tighten this bound, the concept of Minimum Transit Time (MTT) was introduced. The MTT represents the absolute shortest physical time required to enter, traverse, and exit a specific country's transit network. For instance, even under perfect schedule conditions, traveling across Switzerland or Germany requires a mathematically provable minimum number of minutes. The algorithm precomputed this MTT for every country in the dataset.
When calculating the UB, the algorithm sorted all unvisited, reachable countries from the fastest MTT to the slowest (see Algorithm 6 in Appendix X). It then sequentially added these minimum transit times to a `time_spent` counter. The moment the accumulated `time_spent` exceeded the actual hours remaining in the 24-hour journey, the algorithm stopped incrementing the UB. This "Time-Budgeted UB"

drastically accelerated convergence: it accurately pruned paths that possessed theoretical geographic connectivity but lacked the physical time budget required to actually execute the transits.

4.2.3 Priority Scoring, Hard Floors, and Rollouts

To further optimize the search, the standard queue was replaced with a Priority Queue. Rather than expanding blindly, labels were ranked using a custom scoring formula. The formula was designed to prioritize deep, fast routes. For example, a core heuristic evaluated $Score(L) = |C| \times 10000 - t$, effectively forcing the algorithm to aggressively expand labels with the highest country count, while using an earlier absolute time t as the mathematical tie-breaker to favor faster connections (see Algorithm 4 in Appendix X).

Despite prioritized expansion, execution logs revealed the algorithm still wasted resources expanding labels near the end of the timeline that had only accumulated a few countries. To counter this, a Global Hard Floor was introduced (see Algorithm 9 in Appendix X). By establishing a GlobalMinTime (the minimum realistic time to visit any single country), the algorithm calculated the physical time required to bridge the gap between $|C|$ and the target record. If this required time exceeded the remaining clock, the label was killed. Crucially, all arbitrary parameters for these heuristics (such as setting the GlobalMinTime to just 25 minutes) were deliberately defined to be overly optimistic. The architectural philosophy was to tolerate redundant computational overhead rather than risk pruning a mathematically viable, record-breaking route.

To capitalize on the Priority Queue, a Monte-Carlo Rollout mechanism was injected (see Algorithm 8 in Appendix X). Every 1,000 iterations, the algorithm paused standard expansion and launched a rapid, randomized "smart-greedy" probe deep into the graph (see Algorithm 7 in Appendix X). Using heavily biased weights to favor unvisited countries and a Tabu list to prevent local loops, these rollouts operated in milliseconds. If a rollout successfully reached the end of the day and beat the current global record, the new record was dynamically adopted, triggering a massive, immediate pruning wave across the entire Priority Queue.

4.2.4 The Bi-directional Search Pivot

The final algorithmic evolution within this paradigm was born from observing the failure points of the Monte-Carlo rollouts. Debugging revealed that rollouts were highly ineffective when they reached the night hours; physical train schedules became incredibly sparse, trapping the greedy probes in geographic dead-ends.

To bypass this nocturnal sparsity, the monolithic 24-hour search was torn in half. The architecture was rewritten as a Bi-directional Search: one algorithm ran forward in time from the start of the day, while an identical algorithm ran backward in time from the end of the day (see Algorithms 10 and 11 in Appendix X). Both halves executed for 14 hours, creating a 4-hour overlap in the middle of the day.

Finally, a dedicated "Stitcher" algorithm evaluated the intersection of these two sets (see Algorithm 12 in Appendix X). By iterating through overlapping nodes and validating time continuity and geographic unions ($C_{forward} \cup C_{backward}$), the stitcher could reconstruct a globally optimal 24-hour path. This parallel, bi-directional approach effectively bypassed the sparse night-time bottlenecks and maximized graph coverage during the dense daytime schedules.

4.3 The Dimensionality Collapse

Despite the rigorous evolution of the Label-Setting algorithm, from baseline Pareto dominance to time-budgeted Upper Bounds, hard floors, and bi-directional stitching, the architecture ultimately succumbed to the combinatorial complexity of the European transit network. It is crucial to clarify that the algorithm did not fail due to an Out-Of-Memory (OOM) error. Rather, it hit a computational "time wall," suffering a catastrophic degradation in execution speed.

The root of this bottleneck lay in the maintenance of the Pareto frontiers. As the search progressed deeper into the time-dependent multi-graph, the frontiers at each node grew increasingly dense. Because the Guinness World Record routing is fundamentally multi-objective (minimizing time versus maximizing unique countries), vast numbers of paths are mathematically incomparable. A route that visits 8 countries by 14:00 cannot dominate a route that visits 9 countries by 16:00. Consequently, both labels must be kept alive. As these non-dominated sets grew exponentially, the algorithm was forced to perform extensive comparative loops for every newly generated label just to establish dominance, rendering the queue extraction and expansion process computationally intractable.

This inefficiency was heavily compounded by the execution environment. The algorithm was executed locally on a single machine rather than being deployed to a High-Performance Computing (HPC) cluster. This was not a limitation of resources, but a deliberate architectural necessity. Dynamic Programming and Label-Setting algorithms fundamentally rely on continually evaluating and updating a globally shared state, specifically, the dynamic Pareto frontiers at every node in the graph. Attempting to parallelize this specific architecture across distributed cluster nodes (which operate on isolated memory pools) would have introduced massive Inter-Process Communication (IPC) overhead. The constant network latency required to safely lock and synchronize the frontiers across dozens of different parallel workers would have easily neutralized, if not worsened, the computational time. Thus, the approach was inherently bottlenecked by single-thread CPU execution speeds.

After weeks of continuous local execution, the algorithm struggled to converge on any record-breaking paths,a stark contrast to the millions of valid routes that would later be discovered by subsequent methods. It is worth acknowledging that performance could theoretically have been pushed further. By implementing stricter, non-optimistic heuristics (such as aggressively killing technically viable paths to artificially force the queue to empty faster), the algorithm could have achieved faster convergence. However, doing so would have completely sacrificed the exactness of the search. Given the highly specific, non-intuitive nature of train schedules, overly aggressive pruning risked the blind deletion of the optimal, record-winning route.

Having thoroughly tested the boundaries of the Time-Dependent Activity-on-Edge (AoE) graph, the first phase of this comparative study was concluded. The execution bottlenecks clearly demonstrated that while the AoE model achieves exceptional memory efficiency, it imposes an unsustainable temporal and comparative burden directly on the search algorithm. With the exact computational trade-offs of this paradigm established, the analysis now transitions to evaluating the second architectural candidate: the highly parallelizable Activity-on-Node (AoN) framework explored in the following chapter.

5

Algorithmic Approach II: Graph-Based DFS Exploration

6 Results and Conclusions

A
Supporting Material

[Additional tables and data will go here]